

Checklist

Policy-as-Code Implementation

1. Set Up OPA Gatekeeper

- Install OPA Gatekeeper in the Kubernetes cluster.
- Verify Gatekeeper is running correctly.
- Confirm the validating admission webhook is active.
- Identify namespaces that should be excluded from enforcement, such as system namespaces.

2. Define Core Policies

- Create a `ConstraintTemplate` requiring CPU/memory resource requests.
- Require resource limits for all containers.
- Create a policy preventing containers from running as root.
- Prevent privileged containers.
- Require a read-only root filesystem where appropriate.
- Block dangerous Linux capabilities such as `SYS_ADMIN`.
- Restrict container images to approved registries.
- Add clear, human-readable violation messages.

3. Create and Apply Constraints

- Create Constraints from each `ConstraintTemplate`.
- Define which Kubernetes resources each Constraint applies to.
- Exclude required system namespaces.
- Apply the ConstraintTemplates.
- Apply the Constraints.
- Test deployment of a deliberately non-compliant workload.
- Confirm Gatekeeper rejects workloads that violate enforced policies.

4. Adopt Progressive Enforcement

- Begin new policies in audit mode when appropriate.
- Educate developers about why each policy exists.
- Review developer feedback and exemption requests.
- Fix policies that create unnecessary friction.
- Move high-risk policies from audit to enforcement.
- Enforce policies involving genuine security risks.
- Continue monitoring lower-risk standards before enforcing them.

5. Add Shift-Left Testing with Conftest

- Install Conftest.
- Create a `policy/` directory.
- Add Rego policies to the policy directory.
- Run Conftest locally against Kubernetes manifests.
- Verify violations are detected before deployment.
- Add Conftest to developer pre-commit hooks.
- Test the pre-commit validation workflow.

6. Integrate Policy Checks into CI/CD

- Add a policy-validation job to pull-request workflows.
- Install Conftest in the CI environment.
- Run policies against manifests on every pull request.
- Fail the pipeline when required policies are violated.
- Enable GitHub-native output where applicable.
- Confirm violations appear directly in pull-request feedback.

7. Configure Compliance Monitoring

- Enable or verify Gatekeeper auditing.
- Configure Prometheus to scrape Gatekeeper metrics.
- Collect policy-violation data by constraint.
- Collect violation data by namespace.
- Track audit activity.
- Deploy an exporter if additional event-to-metric conversion is required.
- Add error handling and reconnection behavior for production monitoring.

8. Build a Grafana Compliance Dashboard

- Create a Gatekeeper compliance dashboard.
- Add a **Total Violations** panel.
- Add a **Violations by Constraint** panel.
- Add a **Violations by Namespace** panel.
- Add a **Compliance Rate** panel.
- Track remediation progress over time.
- Make the dashboard accessible to relevant stakeholders.

9. Integrate with Broader Governance

- Decide where Gatekeeper should enforce Kubernetes-level policies.
- Identify policies better handled by AWS, Azure, or GCP governance tools.
- Avoid unnecessary cloud-provider lock-in where portability matters.
- Export compliance data to audit or security systems where required.
- Define ownership for policy creation, review, and remediation.

10. Complete the Demo Application Exercise

- Create ConstraintTemplates for resource requests and limits.
- Create security-context policies.
- Create an approved-image-registry policy.
- Copy the Rego policies into a Conftest policy directory.
- Test the demo application's deployment manifest.
- Deploy Gatekeeper to the test cluster.
- Deploy the required Constraints.
- Update the demo application until it passes all policies.
- Create a Python compliance-report script using Gatekeeper/Prometheus metrics.
- Verify the report accurately reflects current compliance status.

11. Optional Developer Portal Integration

- Surface Gatekeeper metrics in Backstage.
- Display current violation count.
- Display the most recent audit time.
- Display policy status.
- Create a service-level compliance scorecard.

Final Validation

- Non-compliant resources are detected before production.
- Critical violations are blocked at admission time.
- Developers receive understandable remediation feedback.
- Compliance posture is visible through dashboards.
- Policies are enforced progressively rather than indiscriminately.
- Exceptions are reviewed instead of becoming permanent workarounds.
- The reason behind every enforced policy is documented and communicated.